arXiv:2207.10838v1 [math.NA] 22 Jul 2022

Quantum-inspired variational algorithms for partial differential equations: Application to financial derivative pricing

Tianchen Zhao* Chuhao Sun* Asaf Cohen* James Stokes†

Shravan Veerapaneni*†

Abstract

Variational quantum Monte Carlo (VMC) combined with neural-network quantum states offers a novel angle of attack on the curse-of-dimensionality encountered in a particular class of partial differential equations (PDEs); namely, the real- and imaginary time-dependent Schrödinger equation. In this paper, we present a simple generalization of VMC applicable to arbitrary time-dependent PDEs, showcasing the technique in the multi-asset Black-Scholes PDE for pricing European options contingent on many correlated underlying assets.

Contents

1 Introduction **2**

2 Theory **3**
2.1 Generalities of McLachlan's variational principle . . . 3
2.2 Analogy with finite element approximations . . . 4
2.3 Neural-network quantum state implementation . . . 4

3 Numerical implementation **4**
3.1 Conversion to meshed form . . . 5
3.2 Autoregressive assumption and sampling . . . 5
3.3 Pre-training . . . 5
3.4 Evolution . . . 6

4 Diffusion with Gaussian Initializations **7**
4.1 Experimental setup . . . 7
4.2 Running Time Analysis . . . 7
4.3 Convergence Visualization . . . 8
4.4 Ablation Study on Batch Size . . . 8

5 Option Pricing **9**
5.1 The Feynmam-Kac representation theorem and option pricing . . . 9
5.2 Option pricing in Black-Scholes model . . . 10
5.3 Numerical Experiments . . . 11

6 Conclusions **12**

7 Acknowledgements **13**

*Department of Mathematics, University of Michigan, Ann Arbor, MI 48109
†Flatiron Institute, Simons Foundation, New York, NY 10010

A Derivation of evolution equations 13

B Matrix representation of $\mathcal{L}$ 14

1 Introduction

The field of stochastic variational algorithms has undergone dramatic recent developments based on two relatively recent, albeit independent, developments: (*i*) the availability of near-term quantum computers [19] and (*ii*) the existence of scalable stochastic algorithms for training deep neural networks. The former development has motivated a new research direction called variational quantum algorithms (VQAs) [6], in which the stochastic variational character of the algorithms render them suitable for noisy intermediate-scale quantum computers [19]. The latter holds promise to accelerate a host of scientific computing problems including general-purpose solvers of partial differential equations (PDEs) using physics-informed neural-networks (PINNs) [20] and has already made substantial headway in solving the time-(in)dependent Schrödinger equation in high dimensions using variational quantum Monte Carlo (VMC) with neural-network quantum states [5]. It is noteworthy that there exist many shared parallels between the fields of VQAs, VMC [25, 24] and PINNs. In particular, both VMC and VQAs hinge on the concept of adaptive stochastic estimation of the time-independent and time-dependent variational principles originally due to Rayleigh-Ritz and McLachlan [18], respectively. VQAs and VMC differ essentially in the choice of parametrized quantum state and the associated adaptive sampling strategy used to estimate quantum expectation values. PINNs can be viewed as an alternative approach to McLachlan's variational principle. They gained significant traction outside the quantum physics literature and have been advocated for solving general PDEs that appear in other areas of science and engineering. This approach involves solving a non-convex optimization problem for the PDE residual in order to determine the space-time development of the state variable. A large number of variants of this proposal have been subsequently put forward targeting PDEs either in many spatial dimensions (e.g., [23, 26]) or parametric PDEs in low dimensions (e.g., [17]). Deep backward stochastic differential equation methods [7, 13, 3, 15] are another class of deep learning approaches that have been applied to parabolic PDEs that arise in mathematical finance.

In this paper, we introduce a generalization of McLachlan's variational principle applicable to a wide variety of time-dependent PDEs and propose a variational quantum Monte Carlo (VMC) stochastic approximate solution method utilizing autoregressive neural-network quantum states. A closely related algorithm has been recently introduced in the VQA literature [8, 1], which was motivated by the problem of solving linear PDEs using digital quantum computers. Ref. [1] argued for an approximate solution concept, with respect to which exponential quantum speedup could be achieved for state evolution, which however is overwhelmed by auxiliary costs of state preparation and extraction of properties of the state (see [1, Section III.C] for details). These complications of read-out and state preparation are not relevant to the VMC-based solver, however, since the VMC computing model permits efficient queries to arbitrary probability amplitudes. Interestingly, our generalization of McLachlan's variational principle is closely related to the neural Galerkin method recently put forward in [4], which also introduced a different neural-network-based stochastic solution method. One small difference compared to [4] is that we choose to work on a predefined mesh, which provides a simple method to implement non-trivial boundary conditions necessary for financial applications. The use of a mesh is not mandatory, however, and this paper paves the way toward a mesh-free generalization. Indeed, in the final stages of preparation of this article, Ref. [21] appeared, which proposes a mesh-free flow-based solution of probabilistic PDEs such as the Fokker-Planck equation. Unlike [21], however, our approach does not require that the state variable of the underlying PDE corresponds to a probability density.

The speedup obtainable by the approach advocated here has practical applications in overcoming the curse-of-dimensionality in high-dimensional PDEs, particularly in situations where fine-grained information about the state variable is required, such as gradients with respect to the independent variables. One such situation is quantum many-body physics, where kinetic energy observables depend on second-order spatial derivatives. Another example is the pricing and hedging of contingent claims in multi-asset financial markets. The approach is applicable to general time-dependent PDEs expressible in first-order form, although we only consider the inhomogeneous linear case in the numerical experiments. Specifically, we chose to focus on the multi-asset Black-Scholes PDE for pricing European contingent claims because of its well-known relation

with the time-dependent Schrödinger equation (TDSE) as well as its importance in computational finance. This work opens the door to free-boundary problems necessary for pricing American contingent claims.

The organization of the paper is as follows: In section 2 we generalize the McLachlan variational principle to general time-dependent PDEs in a model-agnostic manner, which is applicable in the purely classical or quantum setting. In section 3 we describe the modeling assumptions involved in the passage from a time-dependent PDE to an neural quantum state-based solution of McLachlan's variational principle. The remainder of the paper is dedicated to numerical experiments, focusing on the problem of financial derivative pricing, which suffers from a curse-of-dimensionality. Section 4 in particular provides numerical confirmation in the case of correlated diffusions, and section 5 describes the application of these results to option pricing in the Black-Scholes pricing framework.

2 Theory

2.1 Generalities of McLachlan's variational principle

McLachlan's variational principle [18] is an example of a time-dependent variational principle (TDVP) that approximates the solution of the TDSE by evolution within a space of parametrized trial functions. TDVPs for the TDSE have been devised for tensor network states [12], neural-network quantum states (NQS) [5] and parametrized quantum circuits [25, 2]. Ref. [24] studied TDVPs through the lens of information geometry providing a unified perspective applicable to variational quantum algorithms (VQAs) and variational quantum Monte Carlo with normalized neural-network quantum states. In the following section, we further generalize TDVPs to include general time-dependent PDEs and establish additional connections with the VQA and numerical analysis literature. Given a time-dependent PDE for the state variable $u(t,x) \in \mathbb{C}$ with $(t,x) \in [0,T] \times \Omega$, together with a choice of parametrized functions of the spatial variable $\{u_\theta : \Omega \longrightarrow \mathbb{C} \,|\, \theta \in \mathbb{R}^p\}$, the output of a TDVP is parametrized curve $\gamma : [0,T] \longrightarrow \mathbb{R}^p$ in the space of variational parameters such that $u_{\gamma(t)}$ optimally describes $u(t,\cdot)$ in some distance metric $\mathrm{dist}(\cdot,\cdot)$ for all $t \in [0,T]$. Consider the initial value problem for a general time-dependent PDE of the form,

$$\partial_t u(t,x) = \mathcal{F}(t,x,u) \tag{1}$$

$$u(0,x) = u_0 \in L^2(\Omega;\mathbb{C}) \ , \tag{2}$$

with prescribed boundary conditions on $\partial\Omega$. In order to simplify exposition, in this section we avoid complications of boundary conditions by assuming either $\Omega = \mathbb{R}^d$ with suitable decay at infinity or $\Omega = \mathbb{T}^d$. In practice we replace the spatial domain by a mesh $\widehat{\Omega} \subset \Omega$, thereby approximating square-integrable functions by square summable vectors. The imposition of boundary conditions on the spatial mesh is then achieved via the use of source functions. Let Φ_s^t denote the time evolution map for state variable such that $\Phi_u^t \circ \Phi_s^u = \Phi_s^t$ and in particular $u(t,\cdot) = \Phi_0^t(u_0)$. Given an initial parameter vector $\theta_0 \in \mathbb{R}^p$ and a step size $\delta t > 0$, define a sequence of parameter vectors $(\theta_k)_{k\in\mathbb{N}}$ by the following iteration

$$\theta_{k+1} := \underset{\theta\in\mathbb{R}^p}{\arg\min} \left[\mathrm{dist}\left(\Phi_{k\delta t}^{(k+1)\delta t}(u_{\theta_k}), u_\theta \right) \right] . \tag{3}$$

In quantum physics applications, a suitable distance metric is the Fubini-Study metric, as previously argued both in VMC [5, 24] and in VQA [25, 2] literature. In this work we choose $\mathrm{dist}(\cdot,\cdot)$ to be the Euclidean norm. Rather than solving the discrete-time dynamical system (3) directly, we consider the limit of infinitesimal step size $\delta t \longrightarrow 0$ in which it reduces to the following system of ordinary differential equations (ODEs)[1] with initial condition $\gamma(0) = \theta_0$:

$$M(\gamma(t))\,\gamma'(t) = V(t,\gamma(t)) \tag{4}$$

where

$$M_{ij}(\theta) := \mathrm{Re}\left[\left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \frac{\partial u_\theta}{\partial \theta_j} \right\rangle \right] \ , \qquad V_i(t,\theta) := \mathrm{Re}\left[\left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \mathcal{F}(t,u_\theta) \right\rangle \right] \tag{5}$$

and where $\langle\cdot|\cdot\rangle$, $\|\cdot\|_2$ denote the standard inner product and the induced norm for $L^2(\Omega,\mathbb{C})$, respectively. Although the matrix M_{ij} is necessarily positive semi-definite, it may be degenerate, reflecting the possibility

[1] See appendix A for a derivation. Similiar ODEs have been introduced in the neural Galerkin method [4].

of multiple minima in the optimization problem (3). Thus, regularization techniques are generally required in order to obtain a well-posed system of ODEs.

2.2 Analogy with finite element approximations

Before discussing our autoregressive neural-network quantum state implementation, it is useful to orient within scientific computing literature by showing that the formalism shares close parallels with the classic finite element method applied to the linear inhomogeneous case $\mathcal{F}(t,x,u) = \mathcal{L}u(t,x) + f(t,x)$. Given a set of real basis functions $\{\varphi_i\}_{i=1}^m$, define the variational family consisting of a weighted superposition,

$$u_\theta(x) = \sum_{i=1}^{m} \theta_i \varphi_i(x) \ , \tag{6}$$

where $\theta \in \mathbb{R}^m$ is assumed here. Substituting the above into (4) one finds the following ODE determining the dynamics of the weights,

$$M\gamma'(t) = -K\gamma(t) + f(t) \ , \qquad M_{ij} := \langle \varphi_i | \varphi_j \rangle \ , \qquad K_{ij} := -\langle \varphi_i | \mathcal{L} u_j \rangle \ , \qquad f_i(t) := \langle \varphi_i | f(t) \rangle \ , \tag{7}$$

which can be recognized as the standard discrete linear systems arising in finite element methods, with M and K the mass- and stiffness-matrix, respectively. Recall that since finite element techniques use non-overlapping elements to discretize the spatial domain and employ basis functions with localized support, the problem size scales as $m \sim p^d$ in d spatial dimensions, where p is the average number of elements per dimension. Thus, the curse-of-dimensionality arises from need to perform increasingly high-dimensional, albeit sparse, linear algebra in order to solve the linear system (7). The approach advocated in the following section, in contrast, overcomes the curse-of-dimensionality by representing the solution vector in terms of an autoregressive neural-network quantum state.

2.3 Neural-network quantum state implementation

In order to overcome the curse of dimensionality with respect to the spatial dimension d, which is inherent in the utilization of a d-dimensional mesh $\widehat{\Omega} \subset \Omega \subseteq \mathbb{R}^d$, we utilize stochastic estimation combined with autoregressive assumptions. In particular, we take inspiration from both the variational quantum algorithm introduced in [1] as well variational quantum Monte Carlo using autoregressive NQS [22, 14], by parametrizing the solution of the PDE as $u_\theta = \alpha\, \psi_\beta$ where ψ_β is a unit-normalized NQS with variational parameters β and $\alpha > 0$ is a scale factor, whose time-dependence must be determined from the evolution equations along with β. The variational parameters thus consist of the augmented parameter vector $\theta := (\log \alpha, \beta) \in \mathbb{R}^{p+1}$ where $\beta \in \mathbb{R}^p$ is an unconstrained vector representing the weights and biases of the neural network. Plugging the rescaled ansatz into the evolution equations (4), one obtains an augmented system of first-order, non-linear ordinary differential equations determining the time-dependence of the augmented vector θ. The overlaps defining M and V are estimated using the VMC importance sampling technique, where the probability density is chosen to be the modulus-squared wavefunction $|\psi_\beta(x)|^2$.

3 Numerical implementation

In this section, we provide detailed modeling assumptions required to implement the algorithm. After describing a mesh-based encoding of the state variable into a multi-qubit state, we then introduce the model architecture and computation mechanisms including the forward pass and the autoregressive sampling process. Pre-training is necessary in order to satisfy the initial condition of the PDE, and we adopt the standard approach [23] by updating the model iteratively on batches of randomly selected mesh points. Finally, we describe the stochastic estimation procedure used to evolve the variational trial state using McLachlan's variational principle.

3.1 Conversion to meshed form

Rather than working in the continuum, we assume spatial discretization of the function $u(t,\cdot)$ on a regular grid $\widehat{\Omega}$ embedded in the d-dimensional domain $\Omega = [a_1, b_1]\times ... \times [a_d, b_d]$ with a total of 2^n grid points. Without loss of generality, we assume Ω is a regular hypercube satisfying $|b_1 - a_1| = ... = |b_d - a_d|$, and that the mesh size along each axis is $2^{n/d}$. In order to represent the state of the discretized field in terms of the state of an n-qubit system, we assign each computational basis state $|k_1, \ldots, k_n\rangle \in \mathbb{C}^{2^n}$ with $(k_1, \ldots, k_n) \in \{0,1\}^n$ to a linear index defined by $k = \sum_{i=1}^n k_i 2^i$ and then unravel the linear index to indices along each of d axes yielding a d-tuple $(\bar{k}_1, \ldots, \bar{k}_d) \in \{0, \ldots, n/d - 1\}^d$ defined by

$$\sum_{i=1}^{n} \bar{k}_i (2^{\frac{n}{d}})^{i-1} = k. \tag{8}$$

The mesh point $\boldsymbol{x}^k \in \widehat{\Omega}$ corresponding to the linear index $k \in \{0, 1, ..., 2^n - 1\}$ is thus

$$\boldsymbol{x}^k = (a_1 + \bar{k}_1 \Delta x, ..., a_d + \bar{k}_d \Delta x) \tag{9}$$

where $\Delta x = |b_1 - a_1|/2^{\frac{n}{d}}$. Henceforth, we do not distinguish the index k, the binary string $(k_1, \ldots, k_n)$ and the corresponding coordinate $\boldsymbol{x}^k \in \widehat{\Omega}$. The digitized representation of the state of the PDE at time t is thus the following unnormalized n-qubit state

$$|u(t)\rangle = \sum_{k \in \widehat{\Omega}} u(\boldsymbol{x}^k, t) |k_1, \ldots, k_n\rangle \tag{10}$$

and likewise the parametrized approximation $|u_\theta\rangle$ of $|u(t)\rangle$ is given by,

$$|u_\theta\rangle = \sum_{k \in \widehat{\Omega}} u_\theta(\boldsymbol{x}^k) |k_1, \ldots, k_n\rangle \tag{11}$$

where $u_\theta(\cdot)$ is a function defined on $\widehat{\Omega} \subset \Omega$.

3.2 Autoregressive assumption and sampling

In order to obtain an expressive family of trial functions $u_\theta : \widehat{\Omega} \longrightarrow \mathbb{C}$ which furthermore admits an efficient stochastic estimation procedure, we express u_θ as multiple of a unit-normalized neural-network quantum state $\psi_\beta : \widehat{\Omega} \longrightarrow \mathbb{C}$ with variational parameters $\beta \in \mathbb{R}^p$,

$$u_\theta(\boldsymbol{x}^k) = \alpha\, \psi_\beta(k_1, \ldots, k_n) \ , \qquad \|\psi_\beta\|_{\widehat{\Omega}} = 1 \ . \tag{12}$$

A simple method to ensure unit-normalization and efficient sampling is to follow the work of MADE [9] which exploited a masked version of a fully connected layer, where some connections in the computational path are removed in order to satisfy the autoregressive properties, In particular, if we assume a choice of variables such that the state variable is strictly positive[2] $u_\theta(x) > 0$, then a suitable choice of unit-normalized function is

$$\psi_\beta(k_1, \ldots, k_n) = \prod_{i=1}^{n} \sqrt{p_{\beta,i}(k_i | k_{i-1}, \ldots, k_1)} \ , \tag{13}$$

where $p_{\beta,i}(\cdot | k_{i-1}, \ldots, k_1)$ is the parametrized conditional probability distribution for the ith bit.

3.3 Pre-training

Before we perform the time evolution, initial variational parameters θ_0 must be selected in order to match the variational function $u_{\theta_0}(x)$ with the choice of initial condition $u_0(x)$ for $x \in \widehat{\Omega}$ in the spatial domain. This can be achieved via optimization of the following objective function using stochastic gradient descent,

$$J(\alpha_0, \beta_0) = \left\| \alpha_0 |\psi_{\beta_0}\rangle - |u_0\rangle \right\|_{\widehat{\Omega}}^2 \ . \tag{14}$$

[2] This can be considered as a special case of the complex-valued case [22, 14].

3.4 Evolution

In this section we discuss the details of the stochastic estimation of M and V necessary to evolve the augmented parameter vector $\theta = (\log \alpha, \beta) \in \mathbb{R}^{p+1}$. For simplicity we only consider the affine case

$$\mathcal{F}(t, x, u) = \mathcal{L}u(t, x) + f(t, x) \ . \tag{15}$$

Consider the decomposition,

$$M = \left[\begin{array}{c|c} M_{00} & M_{0,1:p} \\ \hline M_{1:p,0} & M_{1:p,1:p} \end{array}\right], \qquad V = \left[\begin{array}{c} V_0 \\ \hline V_{1:p} \end{array}\right]. \tag{16}$$

It is convenient to introduce the following helper functions. In particular, define the Born probability distribution $\rho_\beta(x) \in [0, \infty)$, the wavefunction score $\sigma_\beta(x) \in \mathbb{C}^n$ and the local energy $l_\theta(t, x) \in \mathbb{C}$ as follows,

$$\rho_\beta(x) := |\psi_\beta(x)|^2 \ , \qquad \sigma_\beta(x) := \frac{\nabla_\beta \psi_\beta(x)}{\psi_\beta(x)} \ , \qquad l_\theta(t, x) := \frac{(\mathcal{L}\psi_\beta)(x)}{\psi_\beta(x)} + \frac{f(t, x)}{\alpha} \ . \tag{17}$$

By straightforward calculus, we obtain

$$M_{00} = \alpha^2 \ , \qquad M_{1:p,0} = M_{0,1:p} = \alpha^2 \operatorname{Re}\left[\mathop{\mathbb{E}}_{x\sim\rho_\beta} \sigma_\beta(x)\right], \qquad M_{1:p,1:p} = \alpha^2 \operatorname{Re}\left[\mathop{\mathbb{E}}_{x\sim\rho_\beta} \overline{\sigma_\beta(x)}\, \sigma_\beta(x)^T\right], \tag{18}$$

and

$$V_0 = \alpha^2 \operatorname{Re}\left[\mathop{\mathbb{E}}_{x\sim\rho_\beta} l_\theta(t, x)\right] \ , \qquad V_{1:p} = \alpha^2 \operatorname{Re}\left[\mathop{\mathbb{E}}_{x\sim\rho_\beta} \overline{\sigma_\beta(x)}\, l_\theta(t, x)\right]. \tag{19}$$

The expectation values over x are approximated using Monte Carlo sampling. In practice, the batch of randomly generated samples is represented in the form of a buffer $\mathcal{B} = \{x_i\}_{i=1}^B$ that stores the unique samples in the batch and a counter $\mathcal{C} = \{c_i\}_{i=1}^B$ that records the number of occurrences of each of the corresponding samples. Expectation values are then approximated by sums of the following form,

$$\mathop{\mathbb{E}}_{x\sim\rho_\beta}[g(x)] \approx \frac{1}{\sum_{i=1}^B c_i} \sum_{i=1}^B c_i g(x_i). \tag{20}$$

Computation of per-sample gradient. The computation for both M and V requires direct access to the per-sample gradients $\{\nabla_\beta \psi_\beta(x_i)\}_{i=1}^B$, which is typically not directly accessible during a traditional backward pass by some deep learning software. In order to avoid inefficient forward and backward passes for each of the B samples, we utilize `BackPack` which collects the quantities necessary to compute the individual gradients and reuses them to compute the per-sample gradient without significant computational overhead.

Parallel extraction of matrix elements. Given a sample $x \in \widehat{\Omega}$, corresponding to a particular row-index of the operator $\widehat{\mathcal{L}} \in \mathbb{C}^{2^n \times 2^n}$, the calculation of the local energy $l_\theta(t, x)$ defined in (17) involves determining the nonzero entries of $\widehat{\mathcal{L}}$ in that row. Fortunately, the structure of the PDE problem ensures that the location and values of these entries can be determined in $O(\text{poly}(n))$ time. We exploit CPU parallelization to determine the nonzero row entries for each sample in the batch. Note that the maximal number of nonzero entries per row is determined in advance; e.g. $2n^2 + 1$ for diffusion operator with Dirichlet boundary conditions.

Boundary Conditions. Boundary conditions are implemented on the grid using an appropriate choice of source function. In the case of Dirichlet conditions, for example, we choose the source function as follows

$$f(t, x) = u(t, x) \mathbb{1}_{\partial\Omega}(x) \tag{21}$$

where $\mathbb{1}_{\partial\Omega}$ denotes the indicator function for the set $\partial\Omega$.

Parameter update. The network can be trained by incrementing the parameters using a Euler scheme in the direction $\delta\theta \in \mathbb{R}^{p+1}$ given by the solution of the following linear system

$$M\delta\theta = V\delta t, \tag{22}$$

where M, V are computed as discussed before. In practice, M is usually ill-conditioned due to the fact that it's essentially a sum of B rank one matrices. To stabilize the inverse operation, we consider the singular value decomposition $M = U\Sigma W^T$, and remove the diagonal values of Σ smaller than a small threshold $\epsilon = 10^{-12}$ to obtain Σ_r of size $r\times r$, as well as U_r, W_r of size $(p+1)\times r$. The direction vector is therefore approximated by $W_r\Sigma_r^{-1}U_r^T V\,\delta t$.

4 Diffusion with Gaussian Initializations

In this section, we show various numerical experiments demonstrating the convergence and run time analysis of our proposed approach. Here, we consider the d-dimensional heat equation for which $\mathcal{L} = D\,\nabla\cdot\nabla$ in (15) (diffusion constant set to $D = 0.1$) with either periodic or Dirichlet boundary conditions on $\partial\Omega$. For benchmarking purposes, we employ a finite difference method with the standard central difference scheme to discretize $\mathcal{L}$ (formulas given in Appendix B) and forward Euler for time-stepping.

4.1 Experimental setup

For initialization we chose a discrete isotropic Gaussian, expressed in terms of the modified Bessel function $I_x(t)$ of integer order x,

$$u_0(x) = \prod_{i=1}^{d} e^{-t} I_{x_i}(t) \ , \tag{23}$$

where $t > 0$ is a parameter controlling the width of the Gaussian, and x ranges from 0 to 2^{n-1}. Pre-training was performed using Adam optimizer [16] for 50k iterations with batch size 128, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. The learning rate was warm-started for the first 1/10 total training iterations, then decayed by a factor of 10 at $3/7, 5/7$ of the total training iterations.

After completion of pre-training, the state was evolved for a total evolution time $T = 1$ using a step size of $\delta t = 5\times10^{-5}$ and a batch size of $B = 1024$ for Monte Carlo estimation on each iteration. The time development of the state was compared to the result of Euler time-stepping the initial condition (23) using the same step-size. Due to the exponential scaling of the matrix $\widehat{\mathcal{L}}$, we only establish this baseline for the number of qubits $n \leq 16$. All reported numerical results are averaged across exactly 5 seeds, each trained for 20k steps.

Table 1 reports the relative error between our method and forward Euler method over 20k iterations, for diffusion problem with Dirichlet boundary conditions. The relative error of the obtained solution is computed by comparing the norm of the space-time history of the variational state with the result of Euler development,

$$\text{error} := \frac{1}{T}\sum_{t\in\widehat{\mathcal{T}}} \frac{\left\| |u(t)\rangle - |u_{\gamma(t)}\rangle \right\|_{\widehat{\Omega}}}{\left\| |u(t)\rangle \right\|_{\widehat{\Omega}}} \ . \tag{24}$$

In general, the overall solution obtained using our method matches well with that obtained using forward Euler method. Their discrepancy increases as the dimensionality of the problem increases; nonetheless, within a tolerable threshold (e.g., less than 10%). One remedy is to simply increase the batch size, for example, we show in Section 4.4 that the relative error improves from 15% to around 3% with a batch size that is ten times larger.

4.2 Running Time Analysis

Since we discretize the domain Ω using a grid of size $|\widehat{\Omega}| = 2^n$, the time complexity of forward Euler method, expressed in terms of n is $O(T\times2^{2n})$, where T is the number of iterations. The proposed VMC algorithm,

Table 1: Average relative error of our method in comparison with forward Euler method over 20k iterations. The proposed algorithm solution is compared with a Euler forward method for the diffusion equation over 20k iterations. For forward Euler method, the time step is 5×10^{-5} with a total time of 1. The relative error is computed as $\frac{1}{T}\sum_{t=1}^{T}\frac{\|u(t,x)-f(x;\theta_t)\|}{\|u(t,x)\|}$.

Operator	Boundary Condition	n/d	# of Dimensions d			
			1	2	3	4
Diffusion	Dirichlet	4	5.13×10^{-3}	7.92×10^{-3}	3.12×10^{-2}	1.47×10^{-1}
Diffusion	Dirichlet	5	2.91×10^{-3}	9.91×10^{-3}	7.24×10^{-2}	-

Table 2: Average running time over 2k iterations. The batch size used here is 500. Forward Euler method suffers from the exponential complexity, whereas our method, despite having an overhead running time, enjoys a polynomial scaling. Note that we cannot apply Euler for higher dimensions due to the memory constraint.

Operator	Method	n/d	# of Dimensions d								
			1	2	3	4	5	6	7	8	9
Diffusion	Euler	4	0.024	0.137	1.559	305.92	-	-	-	-	-
Diffusion	Euler	5	0.031	0.225	68.827	-	-	-	-	-	-
Diffusion	Ours	4	29.52	55.76	138.88	230.02	360.80	518.00	792.63	1214.09	1812.94
Diffusion	Ours	5	32.53	88.75	173.43	311.82	507.93	847.85	1355.16	2379.86	4123.61

in contrast, scales as $O(TB\,\mathrm{poly}(n))$, where B is the batch size. In more detail, the forward pass scales as $O(TBn^2)$, and the sampling is $O(TBn^3)$, due to the sequential nature of the auto-regressive sampling process. This polynomial scaling comes at a price of the approximate nature of the time evolution step, and the implicit access to entries of the state vector compared to the forward Euler method which offers $O(1)$ lookup to entries of the state.

In Table 2, we report the average running time of both forward Euler method and our method for 2k iterations (one-tenth of the total running time). The batch size B of our method is fixed to be 500. Note that we cannot apply Euler for higher dimensions due to the memory constraint. Although forward Euler method is effective for small-scale problems, its complexity suffers from the exponential growth with respect to dimension d. The computational cost of our method originates from four sources: sampling, forward pass, per-sample gradient computation (backward pass), and extraction of matrix element information. All these sources contribute to the overhead time that causes our method to run slower in comparison with respect to forward Euler method for problems in lower dimensions. However, as the dimensionality increases, the run time of our method grows only at a polynomial rate. It can be seen from Table 2 that our method is already faster than forward Euler for problem sizes characterized by $n = 16$ qubits.

4.3 Convergence Visualization

We provide snapshots of our method for 2D diffusion problems with periodic boundary conditions over 20k iterations. We run our algorithm with five distinct initializations and record the snapshot every 2k iterations. In particular we do not employ any regularization techniques to enforce that the solution satisfies the boundary condition. It can be observed in Figure 1 that our method successfully obeys the periodic boundary conditions.

4.4 Ablation Study on Batch Size

Recall that M and V are approximated with Monte Carlo sampling using batches of unique samples. Intuitively, a larger batch size yields a better approximation to the exact expectation value, thereby providing more accurate model updates. In this section, we study the effect of batch size on the performance of our method. In the LHS of Figure 2, the running time of our algorithm increases for both larger problem sizes and batch sizes. Note that the actual running time does not grow linearly with respect to the batch size in the plot due to the cache and parallelization. In the RHS of Figure 2, we report the average relative error between the forward Euler method and the VMC method with various batch sizes. Given a fixed problem size (e.g., 4 dimensions with 4 qubits per dimension), we observe a performance improvement by increasing the batch size, which verifies our hypothesis that increasing the batch size does effectively improve

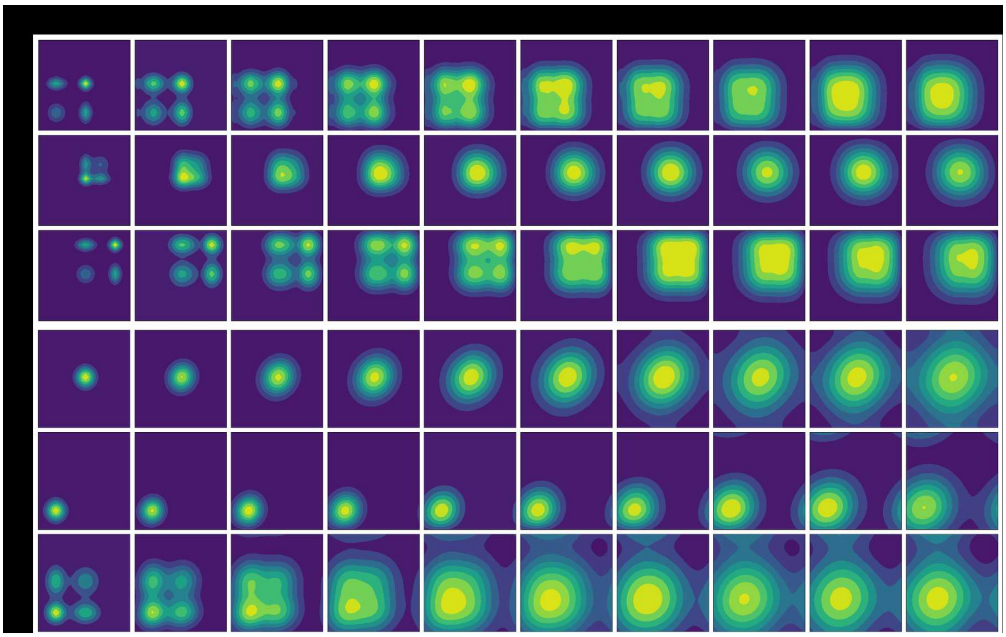

Figure 1: Snapshots of the evolution obtained using our algorithm for diffusion equation with Dirichlet and periodic boundary conditions and different choices of initialization.

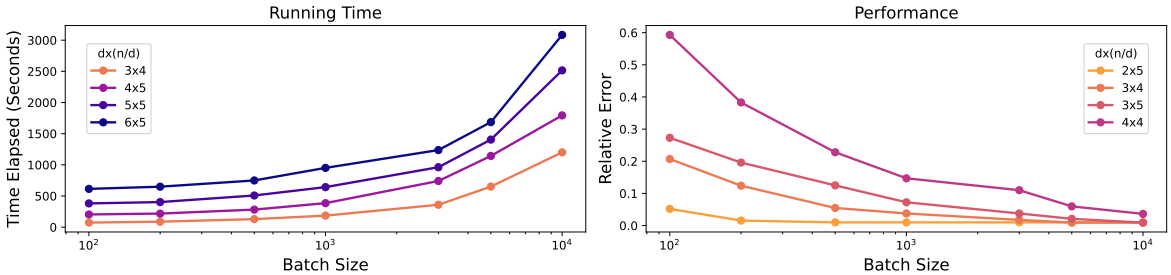

Figure 2: We report the running time and the average relative error with the forward Euler method over various batch sizes, for qubit sizes $n = d \times \frac{n}{d}$, where d is the dimensionality of the problem. The running time grows with respect to the batch size and the problem size. On the other hand, the performance is greatly improved when training with larger batch sizes.

the performance. Given a fixed batch size, our method performs worse as the dimensionality of the problem increases. This result implies that we need a larger batch size to guarantee good performance for problems in higher dimensions.

5 Option Pricing

In this section, we explain how our results can be used to price options. We start with the well-known Feynman-Kac representation theorem and then describe how the numerical results obtained in Section 4 can be applied to the multi-dimensional Black-Scholes model.

5.1 The Feynmam-Kac representation theorem and option pricing

The Feynman-Kac stochastic representation formula links between a parabolic PDE and stochastic differential equations. Namely, consider a PDE of the form:

$$\begin{cases} \frac{\partial u}{\partial t}(t,x) + \mu(t,x) \cdot \nabla_x u(t,x) + \frac{1}{2}\text{trace}[\sigma^T H \sigma](t,x) - r(t,x)u(t,x) + f(t,x) = 0, \qquad (t,x) \in [0,T] \times \mathbb{R}^d, \\ u(T,x) = \Phi(x), \end{cases}$$

where $\mu : [0,T] \times \mathbb{R}^d \to \mathbb{R}^d$, $\sigma : [0,T] \times \mathbb{R}^d \to \mathbb{R}^{d\times d}$, $r, f : [0,T] \times \mathbb{R}^d \to \mathbb{R}$, and $\Phi : \mathbb{R}^d \to \mathbb{R}$ are given measurable functions and H stands for the Hessian matrix of u:

$$H_{ij} = \frac{\partial^2 u}{\partial x_i \partial x_j}.$$

Then, under a second order integrability condition, the solution to this equation takes the form:

$$u(t,x) = \mathbb{E}\Big[\int_t^T e^{-\int_t^\tau r(s,X_s)ds} f(\tau, X_\tau)d\tau + e^{-\int_t^T r(s,X_s)ds}\Phi(X_T)\Big| X_t = x\Big], \tag{25}$$

where $(X_t)_{t\in[0,T]}$ is the solution to the following stochastic differential equation with $(W_t)_{t\in[0,T]}$ being a d-dimensional standard Wiener process:

$$\begin{cases} dX_t = \mu(t, X_t)dt + \sigma(t, X_t)dW_t, \\ X_t = x. \end{cases}$$

One of the many applications of this representation is in derivative pricing. Consider multi-asset with price dynamics $(S_t)_{t\in[0,T]}$ given by the multi-dimension geometric form:

$$\begin{cases} dS_t = \mathrm{Diag}[S_t]\mu(t, S_t)dt + \mathrm{Diag}[S_t]\sigma(t, S_t)dW_t, \\ S_0 = s_0 > 0, \end{cases}$$

where $\mathrm{Diag}[x]$ is a diagonal matrix with the vector x on the diagonal, W is d-dimensional standard Wiener process, $\mu : [0,T] \times [0,\infty) \to \mathbb{R}^d$ and $\sigma : [0,T] \times [0,\infty) \to \mathbb{R}^{d\times d}$ are measurable and satisfy basic conditions that lead to a unique strong solution to the stochastic differential equation above. Consider also the interest rate $r : [0,T] \times [0,\infty) \to \mathbb{R}$. A *risk-neutral measure* is a probability measure $\mathbb{Q}$, such that the discounted price of the asset $(e^{-\int_0^t r(s,X_s)ds} X_t)_{t\in[0,T]}$ is a martingale under $\mathbb{Q}$. The dynamics of the multi-asset under $\mathbb{Q}$ are given by:

$$\begin{cases} dS_t = r(t, S_t)S_t dt + \mathrm{Diag}[S_t]\sigma(t, S_t)dW_t^{\mathbb{Q}}, \\ S_0 = s_0 > 0, \end{cases}$$

where $W_\cdot^{\mathbb{Q}} := W_\cdot - \int_0^\cdot (\mu(t, S_t) - r(t, S_t))dt$ is a standard Wiener process under $\mathbb{Q}$. Then, the price of a simple contingent claim with terminal payment $\Phi(S_T)$ at any given time $t \in [0,T]$, when the multi-asset prices are $S_t = s$ is given by $u(t,s)$, which satisfies the PDE:

$$\begin{cases} \frac{\partial u}{\partial t}(t,s) + r(t,s)s \cdot \nabla_x u(t,s) + \frac{1}{2}\mathrm{trace}[\sigma^T H\sigma](t,s) - r(t,s)u(t,s) = 0, \qquad (t,s) \in [0,T] \times (0,\infty)^d, \\ u(T,s) = \Phi(s), \end{cases}$$

and can be expressed as the conditional $\mathbb{Q}$-expectation as follows:

$$u(t,s) = \mathbb{E}^{\mathbb{Q}}\Big[e^{-\int_t^T r(u,S_u)du}\Phi(S_T)\Big| S_t = s\Big].$$

The most celebrated example is the Black-Scholes model, where the dynamics of the underlying asset follows a geometric Brownian motion and the interest rate is fixed. In dimension 1, the associated PDE admits an explicit solution, which is known as the *Black-Scholes formula*. Aside from the fact that the solution of the PDE is the contingent claim price, its partial derivative $\partial u/\partial x$ is used in order to construct a *delta-hedging* portfolio. Furthermore, its partial derivatives, known as the *Greeks* are used to construct a robust portfolio against small changes, when moving from the continuous- to the discrete-time world.

5.2 Option pricing in Black-Scholes model

We now demonstrate how our algorithm applied to the multidimensional heat equation can be used to price options. For this we consider the Black-Scholes model that consists of a risk-free asset with a constant risk-free return $r > 0$ and d risky assets whose dynamics are given by

$$dS_t^i = \mu_i S_t^i dt + \sigma_i S_t^i dW_t^i, \qquad i = 1, \ldots, d.$$

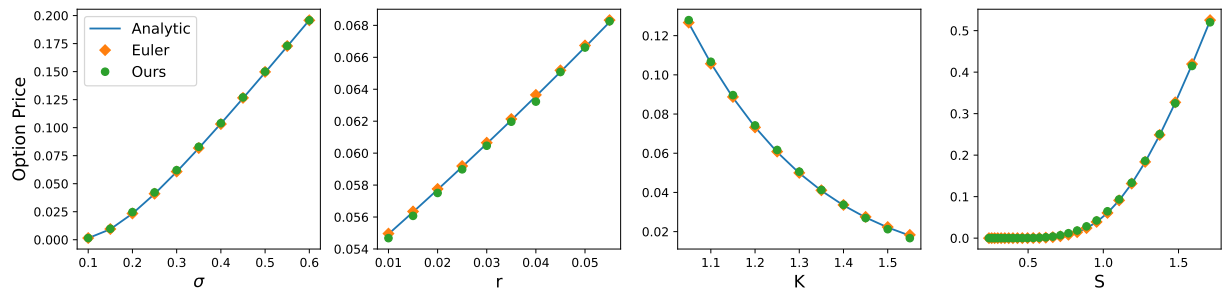

Figure 3: Ablation study on volatility σ, interest rate r, strike price K and initial price. We fix a base setting with hyper-parameters $\sigma = 0.03, r = 0.3, K = 1.25$, and run our algorithm on each setting with only one hyper-parameter deviated from the base setting. In addition, we plot the wave function under the base setting. We compare our solution at the execution time T versus forward Euler method and the corresponding analytic ground truth. Our method is robust under all settings and achieves satisfactory performance.

The parameters μ_i and σ_i, $i = 1, \ldots, d$ are constants, and $\{W^i\}_{i=1}^d$ are Wiener processes with quadratic covariation $[W_t^i, W_t^j] = \rho_{ij} t$.

Consider further a European option, whose payment at (the predetermined) expiry time $T > 0$ is $\Psi(S_T^1, \ldots, S_T^d)$, for some measurable function Ψ. Let V be the conditional price of this option, i.e., $V(t, x_1, \ldots, x_d)$ is the price for the option at time t, given that $S_t^i = x_i$ for $i = 1, \ldots, d$. It is well-known that V satisfies the following Black-Scholes PDE:

$$\begin{cases} \frac{\partial V}{\partial t} + \sum_{i=1}^d r x_i \frac{\partial V}{\partial x_i} + \frac{1}{2} \sum_{i=1}^d \sigma_i^2 x_i^2 \frac{\partial^2 V}{\partial x_i^2} + \sum_{i \neq j} \frac{1}{2} \rho_{ij} \sigma_i \sigma_j x_i x_j \frac{\partial^2 V}{\partial x_i \partial x_j} - rV = 0, & (t, x) \in [0, T] \times (0, \infty)^d, \\ V(T, x_1, \ldots, x_d) = \Psi(x_1, \ldots, x_d). \end{cases}$$

Following Guillaume [10], we may reduce this n-dimensional equation to the n-dimensional standard heat equation. To this end, set

$$u(t, y_1, \ldots, y_n) = V(T - t, e^{\sigma_1 y_1}, \ldots, e^{\sigma_n y_n}) e^{\sum_{i=1}^n -a_i \sigma_i y_i - bt},$$

where a_i and b satisfy the following system of equations:

$$\sum_{i=1}^d a_i \left(r - \frac{\sigma_i^2}{2} \right) + \frac{1}{2} \sum_{i=1}^d a_i^2 \sigma_i^2 + \sum_{i \neq j} \frac{1}{2} \rho_{ij} \sigma_i \sigma_j a_i a_j - r - b = 0,$$

$$r - \frac{\sigma_i^2}{2} + a_i \sigma_i^2 + \sum_{j \neq i} \rho_{ij} \sigma_i \sigma_j a_j = 0, \qquad i = 1, \ldots, d.$$

Then, u satisfies the following heat equation

$$\begin{cases} \frac{\partial u}{\partial t} = \frac{1}{2} \sum_{i=1}^d \frac{\partial^2 u}{\partial y_i^2} + \sum_{i \neq j} \frac{1}{2} \rho_{ij} \frac{\partial^2 u}{\partial y_i \partial y_j}, & (t, y) \in [0, T] \times \mathbb{R}^d, \\ u(0) = \Psi(e^{\sigma_1 y_1}, \ldots, e^{\sigma_d y_d}) e^{-\sum a_i \sigma_i y_i}. \end{cases}$$

with $u(0) = \Psi(e^{\sigma_1 y_1}, \ldots, e^{\sigma_d y_d}) e^{-\sum a_i \sigma_i y_i}$.

5.3 Numerical Experiments

In this section, we apply our algorithm to option pricing in Black-Scholes model across different settings using the numerical solution to the heat equation and the translation from the heat equation to the Black-Scholes equation from the previous subsection. We test the performance of our algorithm as well as show a calculation of the option price for higher dimensions.

In Figure 3 and Table 4 we provide examples to test the performance of our algorithm. In both we include 1D examples for the price of a European call option, whose value at the expiry time T is $V(T, s) = \Psi(s) = \max\{s - K, 0\}$, where K is a predetermined constant, called the strike price. We vary the volatility σ, strike price K, interest rate r, expiry time T, and initial price of the stock S. In Figure 3, we compare

Option Type	Payoff Function at expiry $\Psi(s)$
1D Call	$\max(s-K,0)$
Basket Call	$\max(\sum w_i s_i - K, 0)$
Basket Put	$\max(K - \sum w_i s_i, 0)$
Rainbow Max Call	$\max(\max s_i - K, 0)$
2D Spread Put	$\max(K-(s_1-s_2),0)$

Table 3: Payoff functions for our experiments. We consider basket call and put, Rainbow max call, and spread put options.

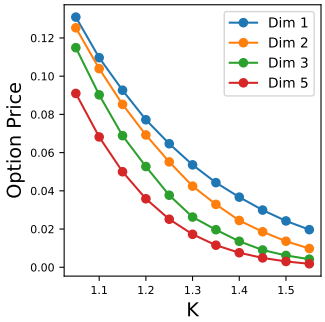

Figure 4: Ablation study on dimensionality, following the base settings in Figure 3, $\sigma = 0.3, r = 0.03, K = 1.25$.

against the forward Euler method and the ground truth *Black-Scholes formula*, which admits an analytical solution in 1D. Specifically, it is given by

$$V(t,s) = N(d_+)s - N(d_-)Ke^{-r(T-t)}, \tag{26}$$

where N is the cumulative distribution function of the standard normal distribution and

$$d_\pm = \frac{\ln\frac{s}{K} + (r \pm \frac{\sigma^2}{2})(T-t)}{\sigma\sqrt{T-t}}.$$

Our method is robust under all settings and achieves satisfactory performance. In Table 4 we compare our accuracy against the forward Euler method, where we use the same analytical solution. Although the performance of our algorithm is inferior to that of Euler, it still achieves good accuracy and the margin can be treated as a price to pay for reducing the exponential scaling down to a polynomial one. Our approach is valuable for high dimensions, where other methods, such as the forward Euler method, suffer from the curse of dimensionality. Table 4 also includes 2D examples with four different options: *Basket European call* and *put*, *rainbow max European call*, and *spread European put*, whose payoffs are listed in Table 3. Note that as we don't have an analytical solution for this case, the relative errors with respect to Euler solutions are reported instead. In Figure 4, we provide a graph for the price of a basket European call option with up to five underlying stocks as a function of the strike price K. As expected the prices are convex with K.

Note that the Black-Scholes PDE lives on the positive orthant of $\mathbb{R}^d$ while its translation to the heat equation lives on $\mathbb{R}^d$. Thereby, all numerical algorithms, including ours, require artificial truncation of the domain. We choose the hypercube domain to be $[s_l, s_u]^d = [Ke^{-3\sigma_i\sqrt{T}}, Ke^{3\sigma_i\sqrt{T}}]^d$. This choice implies that s_l is small (close to $0+$) and s_u is large (close $+\infty$). On the faces of the hypercube, we use the time-discounted payoff functions, as they are reasonably accurate approximations of boundary values of the options considered. Given the number of qubits n and the hypercube input domain for the heat equation $[L_l, L_u]^d$, which is approximately $[-5,5]^d$, the mesh size of each axis is $(L_u - L_l)/(2^{n/d}+1)$.

6 Conclusions

In summary, we introduced a generalization of McLachlan's variational principle applicable to generic time-dependent PDEs as well as a quantum-inspired training algorithm based on neural-network quantum states which can be used to perform approximate time evolution in high dimensions, overcoming the curse-of-dimensionality. Although we focused on a mesh-based formulation in which the quantum state vector is represented by n qubits, it is clear that the mesh is not mandated by the formulation and it would be very interesting to pursue meshless variants based on continuous-variable neural-network quantum states including normalizing flows [24] and to address non-trivial boundary conditions. There exist a number of directions in which the results of this paper can be potentially improved. Since we only considered a first-order Euler approximation of the ODE (4) it would be natural to incorporate high-order time stepping schemes (e.g., Runge-Kutte methods). As an alternative, it would be interesting to pursue a direct solution of the discrete-time dynamical system (3) which has proven successful in both the VMC [11] and VQA [2] literature.

Table 4: List of experiments for the application of our algorithm to Black–Scholes equation. The hyper-parameters for the experiments are listed. We compute the relative error of our method (**Ours**) at expiration time T with analytical ground truth in the 1D case and Euler solution in the 2D case, respectively. For 1D, we also report the relative error of the forward Euler method with respect to the analytical ground truth (**Euler**) for comparison. Our algorithm achieves robust performance across various settings.

Problem	Initial & Boundary Cond.	d	D	T	r	K	σ	Ours	Euler
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.25	0.3	0.011781	0.002494
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.25	0.1	0.032792	0.000930
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.25	0.2	0.017272	0.002389
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.25	0.4	0.011560	0.001392
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.05	0.3	0.086764	0.001750
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.15	0.3	0.013365	0.002521
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.35	0.3	0.012260	0.000328
Black–Scholes 1D	1D CALL	1	-	1	0.03	1.45	0.3	0.012626	0.000845
Black–Scholes 1D	1D CALL	1	-	1	0.01	1.25	0.3	0.010436	0.002538
Black–Scholes 1D	1D CALL	1	-	1	0.02	1.25	0.3	0.010599	0.002515
Black–Scholes 1D	1D CALL	1	-	1	0.04	1.25	0.3	0.014192	0.002477
Black–Scholes 1D	1D CALL	1	-	1	0.05	1.25	0.3	0.017423	0.002463
Black–Scholes 1D	1D CALL	1	-	0.5	0.03	1.25	0.3	0.022481	0.018385
Black–Scholes 1D	1D CALL	1	-	1.5	0.03	1.25	0.3	0.012049	0.014881
Black–Scholes 2D	2D BASKET CALL	2	0.1	1	0.03	1.25	0.3	0.053477	-
Black–Scholes 2D	2D BASKET PUT	2	0.1	1	0.03	1.25	0.3	0.043926	-
Black–Scholes 2D	2D RAINBOW MAX CALL	2	0.1	1	0.03	1.25	0.3	0.057949	-
Black–Scholes 2D	2D SPREAD PUT	2	0.1	1	0.03	1.25	0.3	0.031574	-

7 Acknowledgements

Authors gratefully acknowledge support from NSF under grants DMS-2038030 and DMS-2006305. This research was supported in part through computational resources and services provided by the Advanced Research Computing (ARC) at the University of Michigan.

A Derivation of evolution equations

Assume that the time evolution map and the variational trial function admit Taylor expansions of the form

$$\Phi_t^{t+\delta t}\big(u\big) = u + \mathcal{F}\big(t,u\big)\delta t + \mathcal{O}(\delta t^2) \ , \tag{27}$$

$$u_{\theta+\delta\theta} = u_\theta + \sum_{i=1}^{p} \frac{\partial u_\theta}{\partial \theta_i}\delta\theta_i + \mathcal{O}(\delta\theta^2) \ . \tag{28}$$

Then

$$\begin{aligned}
\left\|\Phi_t^{t+\delta t}(u_\theta) - u_{\theta+\delta\theta}\right\|_2^2 &= \sum_{i,j=1}^{p} \left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \frac{\partial u_\theta}{\partial \theta_j} \right\rangle \delta\theta_i\delta\theta_j - \sum_{i=1}^{p} \left[\left\langle \mathcal{F}(t,u_\theta) \middle| \frac{\partial u_\theta}{\partial \theta_i} \right\rangle + \left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \mathcal{F}(t,u_\theta) \right\rangle \right] \delta t\, \delta\theta_i + \cdots \\
&= \sum_{i,j=1}^{p} \frac{1}{2} \left[\left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \frac{\partial u_\theta}{\partial \theta_j} \right\rangle + \left\langle \frac{\partial u_\theta}{\partial \theta_j} \middle| \frac{\partial u_\theta}{\partial \theta_i} \right\rangle \right] \delta\theta_i\delta\theta_j \\
&\qquad - \sum_{i=1}^{p} \left[\left\langle \mathcal{F}(t,u_\theta) \middle| \frac{\partial u_\theta}{\partial \theta_i} \right\rangle + \left\langle \frac{\partial u_\theta}{\partial \theta_i} \middle| \mathcal{F}(t,u_\theta) \right\rangle \right] \delta t\, \delta\theta_i + \cdots \\
&= \sum_{i,j=1}^{p} M_{ij}(\theta)\delta\theta_i\delta\theta_j - 2\delta t \sum_{i=1}^{p} \delta\theta_i V_i(t,\theta) + \cdots
\end{aligned} \tag{29}$$

where in the last line we used the conjugate-symmetry of $\langle\cdot|\cdot\rangle$ and we have neglected $\delta\theta$-independent terms and terms higher than quadratic order in the multi-variable Taylor expansion in $\delta\theta$ and δt. The first-order

optimality condition $0 = \frac{\partial}{\partial \delta\theta_i} \left\| \Phi_t^{t+\delta t}(u_\theta) - u_{\theta+\delta\theta} \right\|_2^2$, gives, at lowest order in $\delta\theta$ and δt,

$$0 = 2\sum_{j=1}^{p} M_{ij}(\theta)\delta\theta_j - 2V_i(t,\theta)\delta t + \cdots \tag{30}$$

and thus taking the limit $\delta t \longrightarrow 0$ gives the result.

B Matrix representation of $\mathcal{L}$

In d spatial dimensions and multi-index $\mathbf{i} = \{i_1, i_2, \ldots, i_d\}$, let $\mathbf{i} \pm \mathbf{e}_k = \{i_1, i_2, \ldots, i_k \pm 1, \ldots, i_d\}$ and $\mathbf{i} \pm \mathbf{e}_k \pm \mathbf{e}_{k'} = \{i_1, i_2, \ldots, i_k \pm 1, \ldots, i_{k'} \pm 1, \ldots, i_d\}$. Notice we do not allow $+1$ if $i_k = \frac{n}{d}$ or -1 if $i_k = 1$. Then the elements of the matrix is given by:

$$[\widehat{\mathcal{L}}]_{\mathbf{i},\mathbf{j}} = \begin{cases} \frac{-d}{\Delta^2}, & \mathbf{j} = \mathbf{i}, \\ \frac{1}{2\Delta^2}, & \mathbf{j} = \mathbf{i} \pm \mathbf{e}_k, \\ \frac{D}{4\Delta^2}, & \mathbf{j} = \mathbf{i} \pm \mathbf{e}_k \pm \mathbf{e}_{k'}, \\ \frac{-D}{4\Delta^2}, & \mathbf{j} = \mathbf{i} \pm \mathbf{e}_k \mp \mathbf{e}_{k'}, \\ 0, & \text{otherwise.} \end{cases} \tag{31}$$

References

[1] Hedayat Alghassi, Amol Deshmukh, Noelle Ibrahim, Nicolas Robles, Stefan Woerner, and Christa Zoufal. A variational quantum algorithm for the Feynman-Kac formula. *arXiv preprint arXiv:2108.10846*, 2021.

[2] Stefano Barison, Filippo Vicentini, and Giuseppe Carleo. An efficient quantum algorithm for the time evolution of parameterized circuits. *Quantum*, 5:512, July 2021.

[3] Christian Beck, Weinan E, and Arnulf Jentzen. Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations. *J. Nonlinear Sci.*, 29(4):1563–1619, 2019.

[4] Joan Bruna, Benjamin Peherstorfer, and Eric Vanden-Eijnden. Neural Galerkin scheme with active learning for high-dimensional evolution equations. *arXiv preprint arXiv:2203.01360*, 2022.

[5] Giuseppe Carleo and Matthias Troyer. Solving the quantum many-body problem with artificial neural networks. *Science*, 355(6325):602–606, 2017.

[6] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. Variational quantum algorithms. *Nature Reviews Physics*, 3(9):625–644, 2021.

[7] Weinan E, Jiequn Han, and Arnulf Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Commun. Math. Stat.*, 5(4):349–380, 2017.

[8] Filipe Fontanela, Antoine Jacquier, and Mugad Oumgari. A quantum algorithm for linear PDEs arising in finance. *SIAM Journal on Financial Mathematics*, 12(4):SC98–SC114, 2021.

[9] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. MADE: Masked autoencoder for distribution estimation. In *International Conference on Machine Learning*, pages 881–889. PMLR, 2015.

[10] Tristan Guillaume. On the multidimensional Black-Scholes partial differential equation. *Ann. Oper. Res.*, 281(1-2):229–251, 2019.

[11] Irene López Gutiérrez and Christian B Mendl. Real time evolution with neural-network quantum states. *Quantum*, 6:627, 2022.

[12] Jutho Haegeman, J Ignacio Cirac, Tobias J Osborne, Iztok Pižorn, Henri Verschelde, and Frank Verstraete. Time-dependent variational principle for quantum lattices. *Physical review letters*, 107(7):070601, 2011.

[13] Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proc. Natl. Acad. Sci. USA*, 115(34):8505–8510, 2018.

[14] Mohamed Hibat-Allah, Martin Ganahl, Lauren E Hayward, Roger G Melko, and Juan Carrasquilla. Recurrent neural network wave functions. *Physical Review Research*, 2(2):023358, 2020.

[15] Côme Huré, Huyên Pham, and Xavier Warin. Deep backward schemes for high-dimensional nonlinear PDEs. *Math. Comp.*, 89(324):1547–1579, 2020.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations*, 2015.

[17] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

[18] AD McLachlan. A variational solution of the time-dependent Schrodinger equation. *Molecular Physics*, 8(1):39–44, 1964.

[19] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

[20] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[21] Moritz Reh and Martin Gärttner. Variational monte carlo approach to partial differential equations with neural networks. *arXiv preprint arXiv:2206.01927*, 2022.

[22] Or Sharir, Yoav Levine, Noam Wies, Giuseppe Carleo, and Amnon Shashua. Deep autoregressive models for the efficient variational simulation of many-body quantum systems. *Physical review letters*, 124(2):020503, 2020.

[23] Justin Sirignano and Konstantinos Spiliopoulos. Dgm: A deep learning algorithm for solving partial differential equations. *Journal of computational physics*, 375:1339–1364, 2018.

[24] James Stokes, Brian Chen, and Shravan Veerapaneni. Numerical and geometrical aspects of flow-based variational quantum monte carlo. *arXiv preprint arXiv:2203.14824*, 2022.

[25] James Stokes, Josh Izaac, Nathan Killoran, and Giuseppe Carleo. Quantum natural gradient. *Quantum*, 4:269, 2020.

[26] Jiangran Wang, Zhuo Chen, Di Luo, Zhizhen Zhao, Vera Mikyoung Hur, and Bryan K Clark. Spacetime neural network for high dimensional quantum dynamics. *arXiv preprint arXiv:2108.02200*, 2021.